

A
PROJECT REPORT
ON
**FlowFusion: Pseudocode to Python code with
flowchart precision**

Submitted in partial fulfilment of the requirements
of the degree of

Bachelor of Engineering

In
Computer Engineering

by

Sanket Vijay Parulekar 2021016402110992

Patel Nawazish Abdul Raheman 2021016402105503

Amaan Arshad Siddiqui 2021016402107154

Raj Arvind Tiwari 2021016402107193

Supervisor :
Prof. Keerti Kharatmol



Department of Computer Engineering

K.C. College of Engineering and Management Studies and
Research, Thane (E)

University of Mumbai

2024-25

CERTIFICATE

This is to certify that the project entitled **“FlowFusion: Pseudocode to Python code with flowchart precision”** is a bonafide work of **“Sanket Parulekar(B-03)”**, **“Patel Nawazish(B-04)”**, **“Amaan Siddiqui(B-25)”**, **“Raj Tiwari(B-31)”** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **“Bachelor of Engineering”** in **“Computer Engineering”**.

Name and sign

Supervisor/Guide



Dr. Nita Patil

Head of Department

Dr. Vilas Nitnaware

Principal

Project Report Approval for B.E

This project report entitled *FlowFusion: Pseudocode to Python code with flowchart precision* by **Sanket Parulekar(B-03)**, **Patel Nawazish(B-04)** , **Amaan Siddiqui(B-25)** & **Raj Tiwari(B-31)** is approved for the degree of Bachelor of Engineering in **Computer Engineering**.

Examiners

1. _____

2. _____

Date:

Place:

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

Sanket Parulekar (B-03)
Patel Nawazish (B-04)
Amaan Siddiqui (B-25)
Raj Tiwari (B-31)

Date: 19 october 2024

ACKNOWLEDGEMENT

We would like to express special thanks of gratitude to our guide **Mrs. Prof. Keerti Kharatmol** gave us the golden opportunity to do this wonderful project on the topic of **FlowFusion: Pseudocode to Python code with flowchart precision** which also helped us in doing a lot of research and we came to know about so many new things. We are very grateful to our Head of the Department **Dr. Nita Patil** for extending her help directly and indirectly through various channels in our project work. We would also like to thank Principal

Dr. Vilas Nitnaware for providing us the opportunity to implement our project. We are really thankful to them. Finally we would also like to thank our parents and friends who helped us a lot in finalizing this project within the limited time frame.

Thanking You.

TABLE OF CONTENT

Sr.No.	Topic	Page No.
	Certificate	i
	Approval Sheet.....	ii
	Declaration	iii
	Acknowledgement	iv
	List of Figures.....	vi
	List of Table	vii
	Abstract	vii
1.	Introduction.....	1
2.	Literature Survey.....	2
3.	Problem Statement.....	
	3.1 Problem Statement.....	
	3.2 Aim.....	
	3.3 Objectives.....	
4.	Software Requirement Specifications.....	6
	4.1 Requirement Analysis	
	4.1.1 Introduction.....	
	4.1.2 Overall Description.....	
	4.1.3 Functional Requirements.....	
	4.1.4 Non- Functional Requirements.....	
	4.1.5 Activity Diagram.....	
	4.1.6 Gantt Chart.....	
5.	Proposed Work	11
	5.1 Feasibility Study	
	5.2 Project Design	
	5.3 Methodology.....	
	5.3.1 Methodology for pseudocode to flowchart conversion.....	
	5.3.2 Methodology for pseudocode to python code with compilation....	
6.	Conclusion and Future Scope	16
7.	References.....	17

LIST OF FIGURES

Sr. no	Topic	Page no.
4.5.1	Activity Diagram	10
4.6.1	Gantt Chart	11
5.3.1.2	Workflow Diagram of Pseudocode to Flowchart conversion	16
5.3.2.2	Workflow Diagram of Pseudocode to Pythoncode and compilation	17

LIST OF TABLES

Sr. No	Topic	Page No.
2.1	Literature Survey	3

ABSTRACT

FlowFusion is a novel approach designed to bridge the gap between pseudocode and executable Python code, facilitating a streamlined transition from algorithmic design to implementation. This method leverages flowchart precision to enhance the clarity and structure of pseudocode, enabling developers to visualize algorithms effectively before coding. By integrating flowchart elements—such as decision nodes, processes, and data flows—into the pseudocode framework, FlowFusion promotes a better understanding and accuracy in translating logic into Python syntax.

The FlowFusion methodology incorporates systematic techniques for annotating pseudocode with flowchart components, thus ensuring a high level of abstraction while retaining essential details. This allows developers to focus on algorithm design without being bogged down by syntax. Additionally, the process encourages iterative refinement, enabling teams to collaborate more effectively by providing a clear visual representation of the algorithmic flow.

In this Project, we discuss the methodology behind FlowFusion, including its theoretical foundations and practical implementation strategies. We present case studies that showcase how FlowFusion can improve coding efficiency, reduce errors, and enhance overall software quality. Furthermore, we explore the educational implications of FlowFusion, highlighting its potential as a teaching tool for computer science students learning to program.

By promoting a visual and structured approach to coding, FlowFusion aims to empower programmers of all levels, from beginners to experienced developers, to write cleaner, more reliable code. Ultimately, FlowFusion as a transformative tool in software development, bridging the gap between conceptual design and practical implementation while fostering a deeper understanding of algorithms and their applications.

1. INTRODUCTION

In the rapidly advancing field of software development, the ability to effectively translate abstract ideas into executable code is crucial for building robust applications. Pseudocode has long served as a vital tool for developers, allowing them to outline algorithms in a structured, language-agnostic manner. This enables a focus on logic and flow without the distractions of syntax. However, the transition from pseudocode to actual programming languages, such as Python, often presents challenges. Misinterpretations and oversights can arise, leading to inefficiencies and errors in the implementation phase.

To address these challenges, FlowFusion proposes a comprehensive approach that combines the clarity of pseudocode with the visual precision of flowcharts. By incorporating flowchart elements—such as decision nodes, processes, and data flows—into the pseudocode framework, FlowFusion enhances the overall understanding of algorithms. This method allows developers to visualize the logic behind their code more effectively, creating a clearer roadmap from conception to execution.

The integration of flowchart concepts not only streamlines algorithm design but also fosters collaboration among team members, enabling them to engage in discussions with a shared visual reference. As a result, the potential for misunderstandings is significantly reduced, leading to higher-quality code. Furthermore, FlowFusion encourages iterative refinement, allowing developers to revisit and improve their designs before diving into coding.

2. LITERATURE SURVEY

The transition from pseudocode to executable code has been a focal point in computer science education and software engineering for decades. Pseudocode has traditionally been employed as a pedagogical tool, enabling students to outline algorithms without the distraction of syntax. Knuth (1985) emphasized the importance of a structured, language-agnostic format that fosters logical thinking. Dijkstra (1986) and McIver and Conway (2002) further explored how pseudocode can bridge high-level design and low-level implementation, noting that its informal nature can sometimes lead to ambiguity in complex algorithms.

In parallel, visual programming languages and flowchart methodologies have emerged as effective means of representing algorithms. Mayer (2001) demonstrated that visual representations significantly enhance understanding and retention among learners. Flowcharts, specifically, provide an intuitive way to depict control structures and data flows, as highlighted by Harel (1988). Studies by Mott and Guzdial (2008) indicate that integrating flowcharts with programming education can improve problem-solving skills, particularly for novices.

Research in programming education has also focused on strategies to improve learning outcomes and reduce cognitive load. McCracken et al. (2001) found that students engaging with visual representations of algorithms performed better than those learning through traditional text-based methods. Tools that visualize code execution, explored by Kölling (2008), have been shown to enhance understanding and debugging skills.

3. PROBLEM STATEMENT

3.1 Problem Statement:

Developers often face challenges in manually translating pseudocode into Python, which can lead to errors, inefficiency, and lack of clarity. Additionally, creating accurate flowcharts to represent the logic of the code is time - consuming and prone to misinterpretation. The need for a solution that automates both processes accurately converting pseudocode into Python code and generating precise , visual flowcharts remains crucial for improving coding precision and enhancing productivity. FlowFusion addresses this by providing an integrated tool that ensures error - free code conversion , coupled with automated flowchart generation, improving overall workflow and code comprehension.

3.2 Aim:

Efficiently converts pseudocode into Python code while simultaneously designing flowchart for better visualization , enhancing coding precision and development productivity , and empowering users to code faster and with greater accuracy.

3.3 Objectives:

- Develop a user-friendly tool for converting pseudocode to Python code.
- Enhance coding accuracy and efficiency for developers by providing seamless integration of pseudocode, code, and flowcharts.
- Generate flowcharts automatically for visual clarity.

4. SOFTWARE REQUIREMENT SPECIFICATION(SRS)

4.1 Introduction:

4.1.1 Purpose:

The purpose of this SRS document is to provide a detailed description of the system requirements for the "FlowFusion" platform, which integrates flowchart design using binary trees, compiler construction using regular expressions, and shape classification using machine learning algorithms.

4.1.2 Scope:

FlowFusion is a comprehensive tool for data flow design, shape classification, and compiler generation, specifically:

- Uses binary trees to represent and manipulate flowchart designs.
- Leverages regular expressions to facilitate the building of compilers for parsing structured input.
- Employs machine learning algorithms to classify shapes based on input data, potentially for applications such as image recognition or geometric data analysis.

4.1.3 Overview:

This SRS document outlines the product description, functional and non-functional requirements, and constraints for developing the FlowFusion platform.

4.2 Overall Description:

4.2.1 Product Perspective:

FlowFusion is designed as a hybrid tool that facilitates both the design of data flows and the application of advanced algorithms for shape classification and compiler building. The system integrates binary trees to visually represent and design data flowcharts, regular expressions for defining compilers, and machine learning algorithms for classifying shapes based on input data.

4.2.2 Product Functions:

- Binary Tree Flowchart Design: Users can build and manipulate flowcharts using a binary tree structure, which organizes data flow steps and processes.
- Compiler Building Using Regex: Users can define regular expressions to build compilers that parse structured input and generate tokens or abstract syntax trees.
- Shape Classification: Users can upload images or geometric data that will be classified using machine learning algorithms (e.g., Support Vector Machines, Neural Networks).

4.2.3 User Characteristics:

- Software Engineers who need to build compilers.
- Data Analysts and Data Engineers who design flowcharts for data pipelines.
- Machine Learning Researchers and Practitioners who perform shape classification.

4.2.4 Constraints:

- It must be able to scale to include additional shapes and flowchart steps.
- High accuracy in shape classification and flowchart validation is critical.
- Robust error handling for invalid inputs during regex parsing and shape classification.

4.3 Functional Requirements:

4.3.1 Software Requirements:

- Python 3.x or higher.
- Libraries:
 - a. binarytree for binary tree operations.
 - b. graphviz or matplotlib for visualization.
 - c. Pillow for exporting to image formats.

4.3.2 Hardware Requirements:

- Minimum 4 GB RAM.
- Dual-core processor.
- 2–5 GB of available disk space for saving flowchart data and exports
- Operating System: Windows, macOS, or Linux.
- Compatible development environment: An IDE or code editor (e.g., Replit, Python IDLE).

4.3.3 Other:

- The system must allow users to design flowcharts using a binary tree structure
- The system should support operations such as node insertion, deletion, and traversal for managing flowcharts.
- The system should support exporting flowcharts as images (e.g., PNG) .
- The system must allow users to input pseudocode for conversion into Python code.
- The system should compile the generated Python code and report any syntax errors or execution issues.

4.4 Non - Functional Requirements:

4.4.1 Performance:

- The system should handle up to 1000 nodes in flowcharts without significant delays.
- Response time for shape classification should be under 2 seconds for images.

4.4.2 Security:

- The system must implement user authentication for saving/loading flowchart data and compiled outputs.
- Regular expressions must be sanitized to prevent injection attacks.

4.4.3 Availability:

- The system should be available for use 24/7, with regular backups to prevent data loss.
- Scheduled maintenance must be communicated to users in advance.

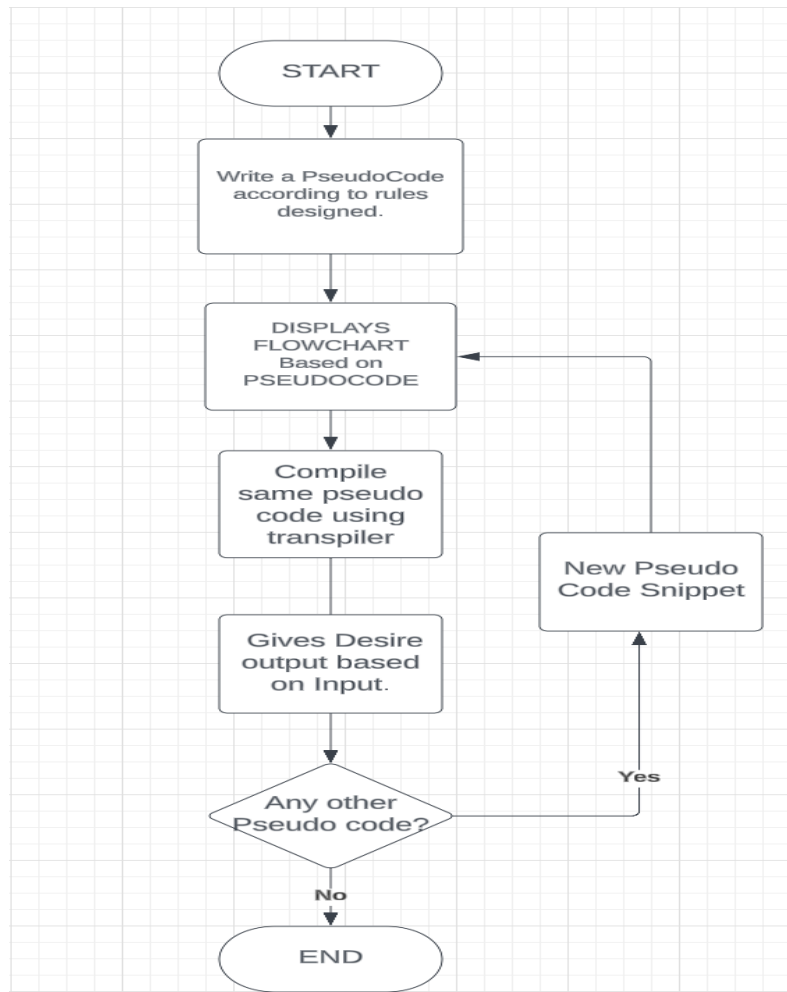
4.4.4 Maintainability:

- The codebase must follow established coding standards and documentation practices to facilitate future maintenance.
- Automated tests should cover critical functions to ensure ongoing reliability during updates.

4.4.5 Reliability:

- The system should maintain 99.9% uptime to ensure continuous availability for users.
- Error logging must capture all critical failures to assist in troubleshooting.

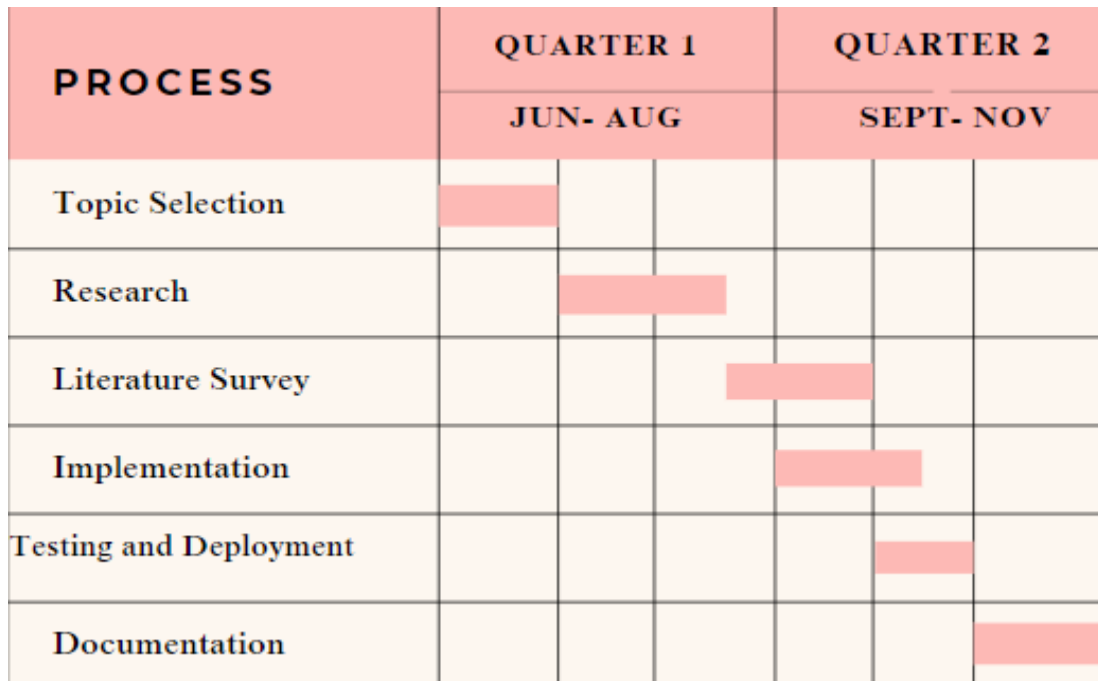
4.5 Activity Diagram:



4.5.1 Activity Diagram of FlowFusion

The activity diagram illustrates the flow of the FlowFusion system, starting with the user writing pseudocode according to the designed rules. Upon writing the pseudocode, the system displays a corresponding flowchart. The pseudocode is then compiled using a transpiler, providing the desired output based on user input. The diagram concludes by checking if the user wishes to input another pseudocode snippet, looping back to allow for additional entries if desired, or ending the process if not.

4.6 Gantt Chart:



4.6.1 Gantt chart

The Gantt chart outlines a project timeline across two quarters, from June to November. Topic selection takes place during June and July, followed by research spanning July to August. The literature survey, implementation, and testing phases occur sequentially from September to November. The process concludes with documentation in November.

5. PROPOSED METHEDOLOGY

The FlowFusion system is designed to convert high-level pseudocode into precise Python code while ensuring the generated code aligns perfectly with the logical flow of a flowchart. The process begins with the system accepting pseudocode as input. This pseudocode is analyzed to understand its structure, including control flows like loops, conditionals, and function calls. To achieve this, the system will utilize natural language processing techniques, allowing flexibility in input style, ensuring that even loosely formatted pseudocode is correctly interpreted.

5.1 Feasibility study

- Technical Feasibility:

Natural Language Processing (NLP) can effectively parse and interpret pseudocode using existing libraries like spaCy or NLTK, transforming informal syntax into structured input. For flowchart generation, tools such as Graphviz or Diagrams can visually represent the parsed pseudocode, ensuring coherent and accurate diagrams. Additionally, Python's clear syntax rules make it feasible to automate code generation from pseudocode, allowing for the conversion of logical elements like loops, conditions, and function calls into appropriate Python constructs.

- Operational Feasibility:

The system will feature a user-friendly interface that allows developers to input pseudocode and receive corresponding Python code and flowcharts, making it operationally viable for real-world use. It will include error handling to manage incomplete or inconsistent inputs, providing user feedback through validation mechanisms during parsing. Additionally, FlowFusion will minimize the learning curve, enabling developers to automate code generation without the need to master new tools or languages. continue writing pseudocode in their preferred style, and the system will take care of the translation.

- Economic Feasibility:

Initial development costs for the system will focus on creating the NLP parsing engine, flowchart generation module, and code generator, but existing libraries and frameworks will help reduce both time and expenses. In the long term, automating the conversion of pseudocode to Python code can significantly lower development costs and time, particularly for large-scale projects. By aligning the generated code with the intended logical flow, this solution also minimizes the need for debugging and refactoring, further enhancing efficiency.

5.2 Project Design

FlowFusion aims to convert pseudocode into Python code while aligning with the logical precision of flowcharts. The system's design integrates various technologies to achieve high accuracy in parsing, flowchart generation, and code conversion. The design can be broken down into the following components, each employing specific technologies to accomplish the objectives.

- Data Structures: Binary Tree for Logic Representation:

Binary trees will represent the logical flow of the pseudocode, providing an efficient structure for parsing and code generation. After parsing, conditional branches, loops, and function calls will be organized in binary nodes, where the left node corresponds to the if branch and the right node to the else branch. This binary tree will serve as the foundation for generating flowcharts and synthesizing Python code, facilitating seamless transitions between logical structures.

- Machine Learning Algorithms for Shape Classification in Flowchart Generation:

Machine learning algorithms will classify shapes in flowcharts to ensure accurate identification and rendering of components like processes, decisions, and inputs/outputs during flowchart generation. The model will be trained to recognize various shapes such as rectangles, diamonds, and parallelograms based on the logical structure in the binary tree. This classification enables the system to accurately visualize the flowchart, reflecting the logic of the parsed pseudocode.

- Regular Expressions for Compiler Design:

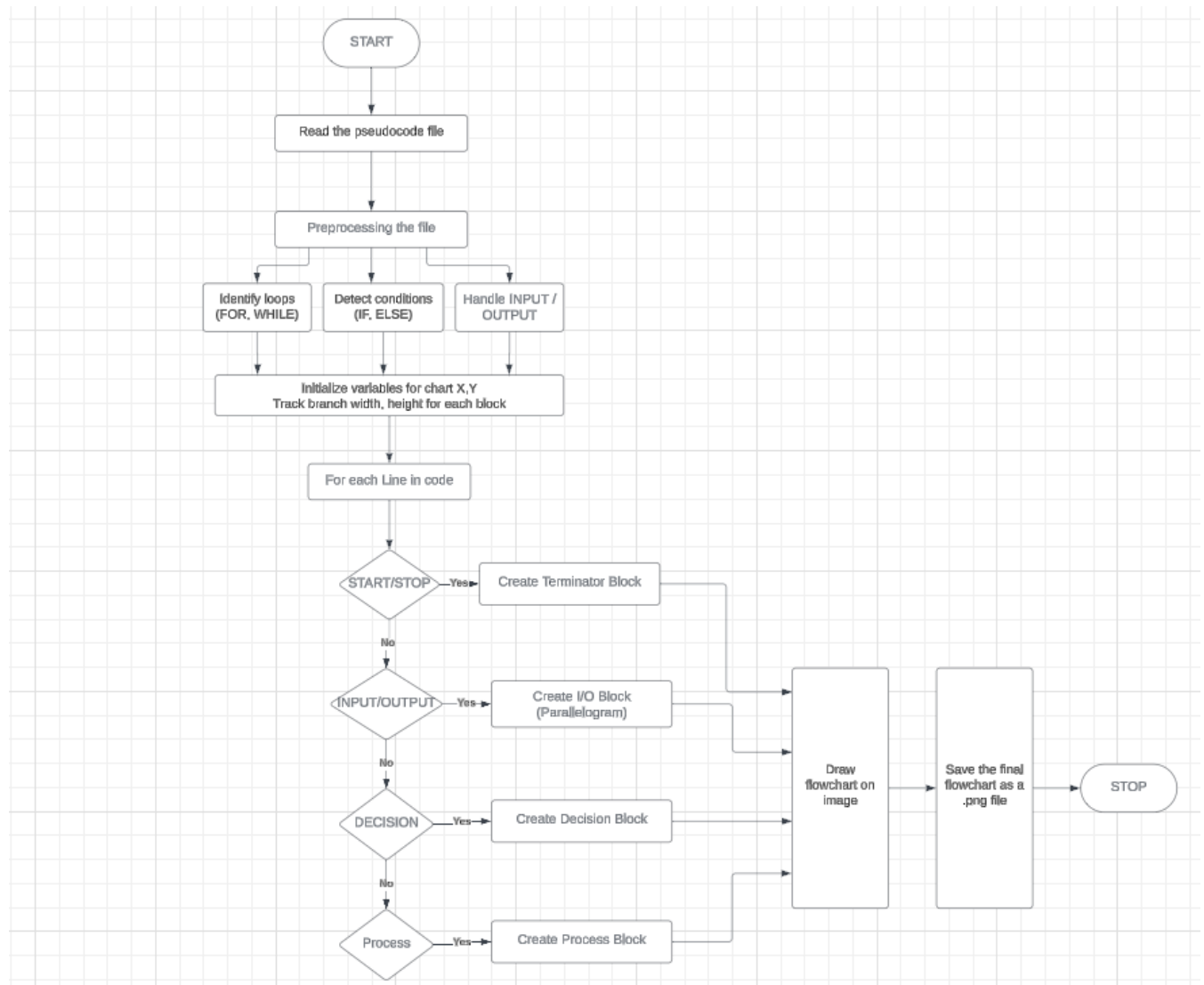
Regular expressions will be utilized in the compiler design to interpret and tokenize pseudocode, transforming unstructured input into a format suitable for binary tree representation. During the lexical analysis phase, they will break down the pseudocode into meaningful tokens, such as keywords, operators, and identifiers. This structured representation ensures accurate mapping of pseudocode components, like loops and conditions, to the corresponding binary tree nodes.

5.3 Methodology

5.3.1 Methodology for Pseudocode to Flowchart Conversion:

1. Algorithm:

- i. Start the process.
- ii. Read the pseudocode file.
- iii. Preprocess the file.
- iv. Identify loops (FOR, WHILE).
- v. Detect conditions (IF, ELSE).
- vi. Handle INPUT/OUTPUT.
- vii. Initialize variables for chart X, Y.
- viii. Track branch width and height for each block.
- ix. For each line in the code, check if it's START/STOP.
 - x. If START/STOP, create Terminator block.
 - xi. If INPUT/OUTPUT, create I/O block (Parallelogram).
 - xii. If DECISION, create Decision block.
 - xiii. If none of the above, create Process block.
- xiv. Draw the flowchart on the image.
- xv. Save the final flowchart as a .png file.
- xvi. Stop the process.

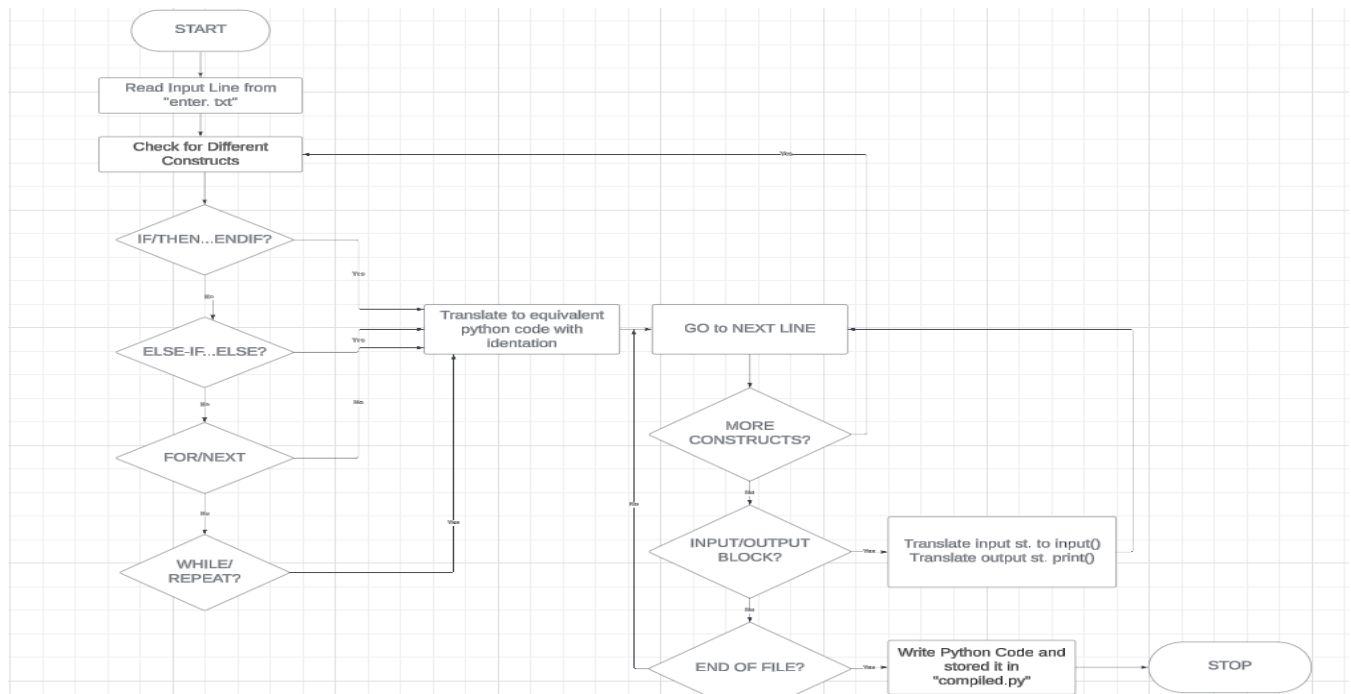


5.3.1.2 Workflow Diagram for Pseudocode to Flowchart Generation

5.3.2 Methodology For Pseudocode to Python code with Compilation

1. Algorithm:

- i. Start the process.
- ii. Read input line from "enter.txt".
- iii. Check for different constructs.
- iv. If IF/THEN...ENDIF, translate to equivalent Python code with indentation.
- v. If ELSE-IF...ELSE, translate to equivalent Python code with indentation.
- vi. If FOR/NEXT, translate to equivalent Python code with indentation.
- vii. If WHILE/REPEAT, translate to equivalent Python code with indentation.
- viii. Go to the next line.
- ix. Check if there are more constructs.
- x. If an INPUT/OUTPUT block, translate input statements to input() and output statements to print().
- xi. If end of file, write the Python code and store it in "compiled.py".
- xii. Stop the process.



5.3.2.2

Workflow Diagram for Pseudocode to Python code with compilation

6. CONCLUSION

Conclusion:

The conclusion of the FlowFusion project highlights its importance in improving coding efficiency and visual clarity for developers. The tool effectively converts pseudocode into Python code and automatically generates corresponding flowcharts, streamlining the development process. The project emphasises creating a user-friendly environment that allows for precise and integrated conversion and visualisation, making coding more accessible and productive for users.

REFERENCES

1. Abhay Chopade, Vaibhav Shingade, Aman Chavare, Tejas Bhagwat. "**Code Insight - Flowchart Generator.**" 2024. The project automates pseudocode conversion into flowcharts using OpenAI GPT-3.5 Turbo.
2. Shreya Shukla, Prajwal Gatti, Yogesh Kumar, Vikash Yadav, Anand Mishra. "**Towards Making Flowchart Images Machine Interpretable.**" 2023. Model: FloCo-T5, a transformer-based framework.
3. Damitha Karunratna, Nasik Shafeek. "**An Extensible Approach to Generate Flowcharts from Source Code.**" 2018. Language-independent abstract representation.
4. Xiangyu Zhang, Jianjun Zhao, Haipeng Cai, and Ruoyu Xu. "**Automatic Translation from Pseudocode to Source Code: A Machine Learning Approach.**" 2021.
5. ikhail Denisov, Anton Anikin, Oleg Sychev. "**Dynamic Flowcharts for Enhancing Learners' Understanding of Control Flow During Programming Learning.**" 2021. Implementation of a non-editable flowchart widget.